Vyuham — Extension: Flood Extent & Damage Layer (Study C)

Addendum to the *Kerala Climate Network — Vyuham Build Spec*. Do this **after** the core pipeline and dashboard (Studies A and B) are complete. Nothing here changes the core design; it adds one new data layer, three pipeline stages, new fields in the data contract, and one dashboard page.

1. Purpose and research questions

The core spec studies the rainfall network and validates it against 2018 landslides. It does not say where flooding actually happened. This extension adds that.

- **C1 — Flood:** Were the cells and basins that are most central in the extreme-rainfall network also the most flooded in August 2018?
- **C2 — Damage:** Does centrality relate more strongly to *damaging* landslides than to landslides in general?
- **C3 — Baseline:** Does the network add anything beyond simple rainfall totals for explaining flooding? (This is the check that makes C1 meaningful.)

All three are exploratory: one event, a small number of cells and basins, and no causal claim.

2. Go / no-go checkpoint

Start the extension only if, at the end of the core build, all of these are true:

- `make all` runs from scratch and the core dashboard works.
- `cells.parquet`, `metrics.parquet`, the basin assignment and the landslide table exist.
- You have at least **2 free days** left.

If not, list this extension as future work in the report. A finished core is worth more than a half-finished extension.

3. Prerequisites (do these early, ideally on Day 1 of the core build)

1. **Google Earth Engine account.** Register a free (non-commercial) account and confirm you can run a Python or Code Editor script. Approval can take time, so do not leave this until later.
2. **Inspect the landslide inventory's damage fields.** Open the 2018 inventory (van Westen 2020) and check which columns record damage, and how many rows are filled in. The GSI–KSDMA part of the inventory was built from a survey of landslides that damaged buildings, roads and other infrastructure, and the edited inventory carries attributes such as failure type, size and damages. How complete these are is unverified, so look before you plan around them.
3. **Inspect the Kaggle "Kerala Floods 2018" dataset.** It is listed as containing warning, fatality, relief-camp and rainfall data for the 2018 flood. Check whether it is district-level or finer. If district-level, it can support a summary chart but not a cell-level map.
4. **Note the core outputs you will join to:** `cell_id`, basin assignment, centrality columns, landslide counts and slope.

4. Data sources

Layer	Source	Used for	Notes and risks
Flood extent, Aug 2018	Sentinel-1 SAR (VV) via Google Earth Engine	Flooded area per cell and per basin	Radar sees through cloud, which matters in monsoon season. Check the current collection ID in the Earth Engine catalog
Non-flood reference	Sentinel-1 from a dry period and from earlier Augusts	Change detection baseline	Published Kerala work compared flood dates with January 2018 and with August of previous years
Permanent water mask	JRC Global Surface Water (Earth Engine), as far as I know	Remove rivers, lakes and backwaters	Confirm the dataset and version before use
Slope / elevation	SRTM 30 m (already in the core pipeline)	Mask steep terrain; stratify by elevation	Reuse the core DEM
Damaging landslides	2018 inventory (already in the core pipeline)	C2	Depends on the damage columns you found in prerequisite 2
District damage indicators	Kaggle Kerala Floods 2018 (optional)	Rough district-level summary	Not a map; verify spatial detail first

5. Method: new pipeline stages

These continue the numbering of the core spec (stages 1–8). Keep every parameter in `config.yaml`.

Stage	What it does	Output
9 Flood mask	In Earth Engine: build Sentinel-1 composites for the flood dates and for the reference periods, apply speckle smoothing, detect the backscatter drop that marks open water (an automatic threshold such as Otsu has been used for Kerala 2018), then remove permanent water and steep terrain. Do the aggregation **inside Earth Engine** and export only per-cell and per-basin flooded area as a table, not a huge raster	`flood_by_cell.csv`, `flood_by_basin.csv`, optional coarse flood layer for display
10 Damage layer	Filter the inventory to landslides with recorded damage (using the columns from prerequisite 2). Count per cell and density per km². If usable, load district-level indicators from the Kaggle file for a summary chart	`damage_by_cell.parquet`
11 Statistics	Spearman correlation of each centrality vs `flooded_fraction`, overall and stratified by elevation class (lowland / midland / highland). **Baseline:** the same correlation for total Aug 2018 rainfall and for extreme-day count, then check whether centrality adds explanatory value beyond them (for example partial rank correlation or a two-variable rank regression). Repeat for damaging-landslide density vs all-landslide density. Basin level: rank correlation between basin impact rank and basin flooded fraction, reported descriptively because the number of basins is small. Bootstrap confidence intervals throughout	`flood_stats.parquet`
12 Export	Write the new fields and files listed in section 6	`web/public/data/*`

Flood dates. Published Kerala work used Sentinel-1 scenes around 9 and 21 August 2018, and the Kuttanad lowlands were near peak inundation around 18 August. Use dates within that window and keep them in `config.yaml`.

Sanity checks for the flood mask (the exact numbers come from other studies, not from your run):

- The Kuttanad flood extent has been reported at roughly 520 km² at peak (18 August 2018) from SAR data. Your mask should land in the same order of magnitude for that region.
- Flooded fraction must fall between 0 and 1 for every cell.
- A January or other dry-period mask should show almost no flooded area outside permanent water.
- Compare visually against a published inundation map for at least one region.

6. Data contract additions

New or changed fields (add these to the core contract first, then implement both sides):

```
cells.geojson    + flooded_area_km2, flooded_fraction, mean_elevation,
                   elevation_class, damaging_landslide_count,
                   damaging_landslide_density, aug2018_rain_total

basins.geojson   + flooded_area_km2, flooded_fraction, flood_rank

flood_stats.json { overall: { metric: {rho, p, ci_low, ci_high} },
                   by_elevation_class: [ {class, n, metric, rho, p} ],
                   baseline: { rain_total: {rho, ci}, extreme_days: {rho, ci},
                               network_added_value: {...} },
                   scatter: [ {cell_id, centrality, flooded_fraction, elevation_class} ],
                   damage: { damaging: {rho, ci}, all_landslides: {rho, ci} } }

flood_layer.geojson   optional, simplified flood polygons or a coarse grid;
                      keep under ~5 MB

meta.json        + flood_dates, sensor, threshold_method, water_mask_source
```

7. Dashboard additions

- **New page: Flood & Damage.** Map with three toggles (centrality layer, flood layer, damaging landslides). Scatter of centrality vs flooded fraction coloured by elevation class. A "baseline vs network" card comparing the rainfall-total correlation with the centrality correlation. Basin table with impact rank next to flood rank.
- **Landslide page.** Add a toggle: all landslides vs damaging landslides only.
- **Overview.** One extra stat card (strongest flood correlation).
- **Method & limits.** Add the limits in section 9.

8. Repository additions

```
gee/flood_mask.py          # Earth Engine script (or notebook) for stage 9
src/ktnet/flood.py         # zonal joins, flooded fraction, baseline comparison
src/ktnet/damage.py        # damaging-landslide filter and per-cell counts
pipeline/09_flood.py
pipeline/10_damage.py
pipeline/11_flood_stats.py
tests/test_flood.py        # fraction bounds, dry-period check, join integrity
web/src/pages/FloodDamage.tsx
```

9. Limits to state

- **SAR can under-detect flooding** under dense vegetation, in built-up areas and in radar shadow on hills, so flooded fraction is a lower bound.
- **One event.** Results describe August 2018 only.
- **Rainfall is not the only driver.** Flooding also depends on terrain, drainage and reservoir operations, none of which the rainfall network sees.
- **Cell size.** At about 27 km, one IMD cell covers many different local flood conditions, so cell-level flooded fraction is an average.
- **Small samples.** There are only a handful of basins, so basin-level statistics are descriptive.
- **Correlation, not causation.**

10. Timeline (2–3 extra days)

Day	Work	Done when
E1	Earth Engine flood mask, per-cell and per-basin export, sanity checks against published numbers	`flood_by_cell.csv` passes the checks in section 5
E2	Damage layer, statistics including the rainfall baseline, export new contract files	`flood_stats.json` and updated `cells.geojson` validate
E3	Flood & Damage page, Landslide toggle, Method page update, README and screenshots	Full dashboard builds with the new page

Cut lines: if the flood mask is not reliable by the end of E1, skip the flood map and do only the damage comparison (C2). If E2 runs long, report correlations without the elevation stratification.

11. Prompts for the coding agents

Flood mask: "Read the extension spec, section 5, stage 9. Write an Earth Engine Python script that builds Sentinel-1 VV composites for the dates and reference periods in `config.yaml`, detects flooded pixels by backscatter change, masks permanent water and steep terrain, and exports flooded area per IMD grid cell and per basin as CSV. Do not download rasters. Print total flooded area for the Kuttanad region so I can check it."

Statistics: "Implement stage 11 in `src/ktnet/flood.py`. Join flood and damage tables to the cell table, compute Spearman correlations with bootstrap confidence intervals, add the rainfall-total and extreme-day baselines, and write `flood_stats.json` following the contract. Add tests for bounds and for join integrity."

UI page: "Add the Flood & Damage page from section 7 using the new contract fields. Generate mock `flood_stats.json` and updated `cells.geojson` first so the page works before the real data exists."

12. Sources to check before citing

Verify authors, years and journals yourself before using any of these in a report.

- Flood inundation mapping of Kerala 2018 with Sentinel-1 SAR, an automatic threshold and Google Earth Engine (open access, PMC).

- Vanama et al., inundation mapping of the Kerala 2018 flood using ALOS-2 and temporal Sentinel-1 SAR.
- Catastrophic flood of August 2018, Kerala: role of lineaments in modulating flood level, using remote-sensing data (ScienceDirect).
- Hao et al. (2020), a complete landslide inventory for the 2018 monsoon disaster in Kerala, *Earth System Science Data*.
- FL-MHSM (arXiv 2604.16265): flood and landslide multi-hazard susceptibility mapping, which also builds Kerala flood inventories from Sentinel-1 in Earth Engine.